Proofs are Programs

IMP - Simple imperative programs

Simple Imperative Programs

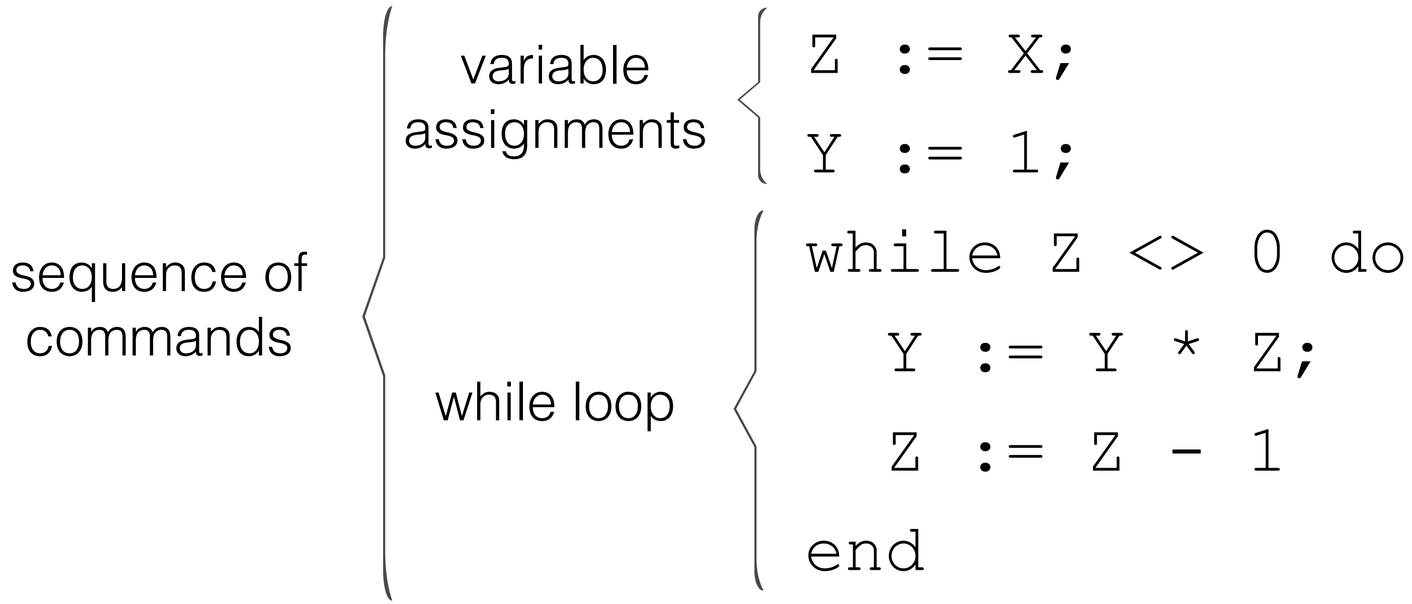
```
Z := X;
Y := 1;
while Z <> 0 do
  Y := Y * Z;
  Z := Z - 1
end
```

Simple Imperative Programs

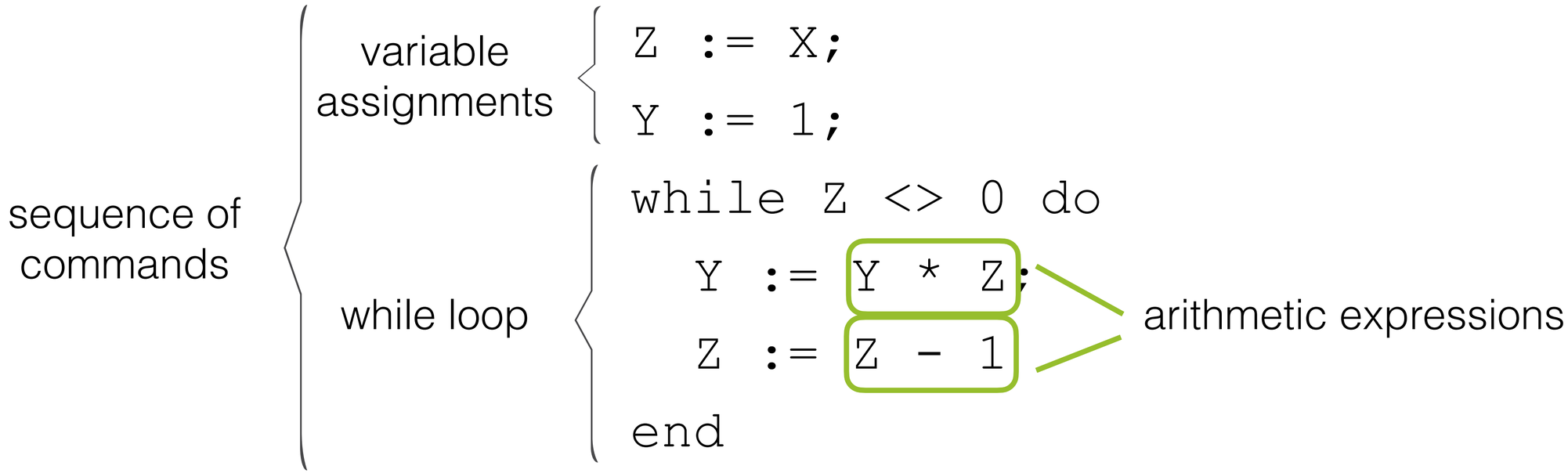
sequence of commands

```
Z := X;
Y := 1;
while Z <> 0 do
  Y := Y * Z;
  Z := Z - 1
end
```

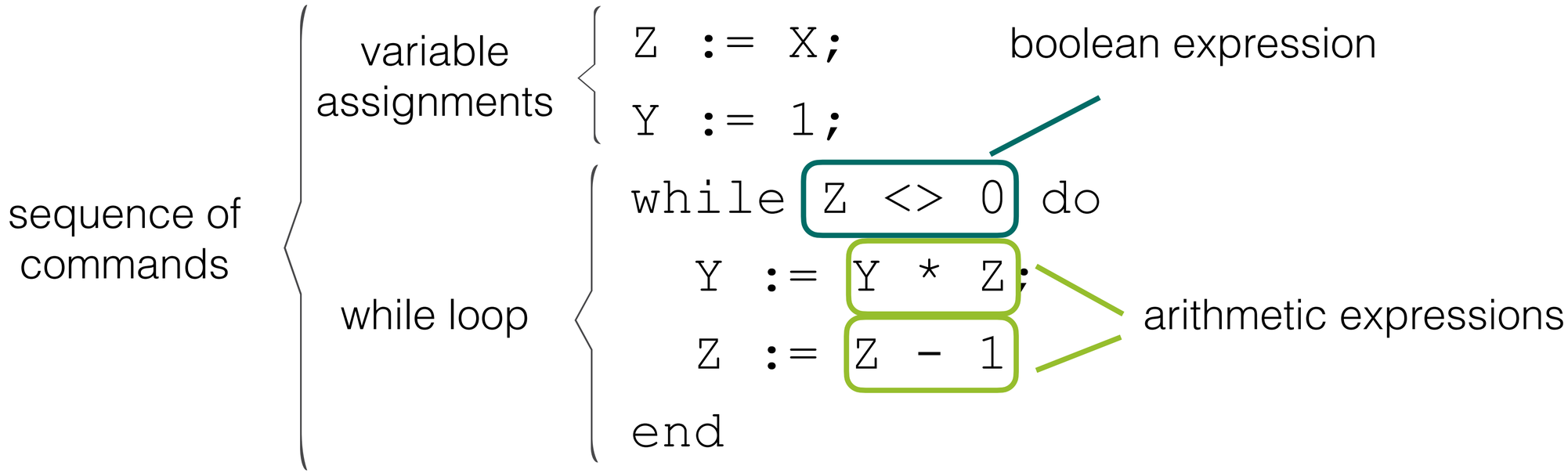
Simple Imperative Programs



Simple Imperative Programs



Simple Imperative Programs



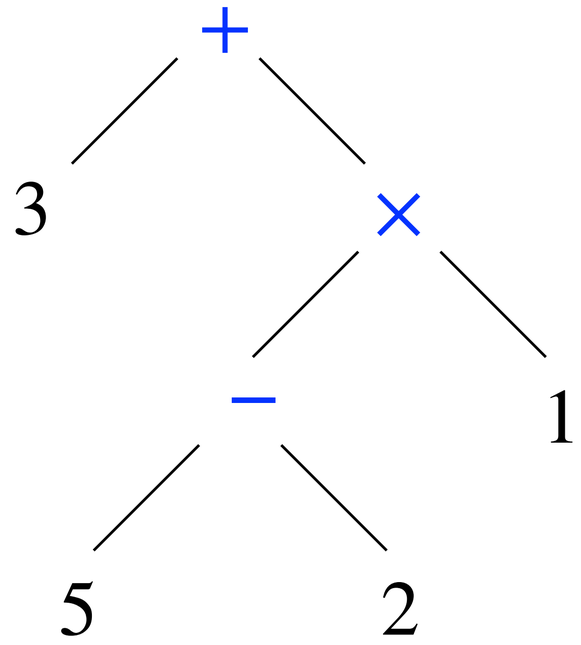
Expressions

Arithmetic Expressions (BNF)

$$
\begin{aligned}
a &:= n \\
&\mid a + a \\
&\mid a - a \\
&\mid a \times a \\
n &\in \mathbb{N}
\end{aligned}
$$

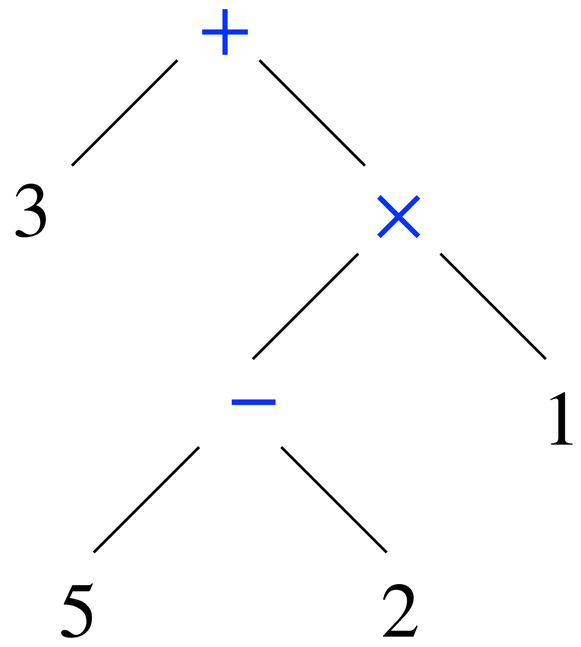
Arithmetic Expressions (BNF)

$$
\begin{aligned}
a &:= n \\
&\mid a + a \\
&\mid a - a \\
&\mid a \times a \\
n &\in \mathbb{N}
\end{aligned}
$$



Arithmetic Expressions (BNF)

$$
\begin{aligned}
a &:= n \\
&\mid a + a \\
&\mid a - a \\
&\mid a \times a \\
n &\in \mathbb{N}
\end{aligned}
$$



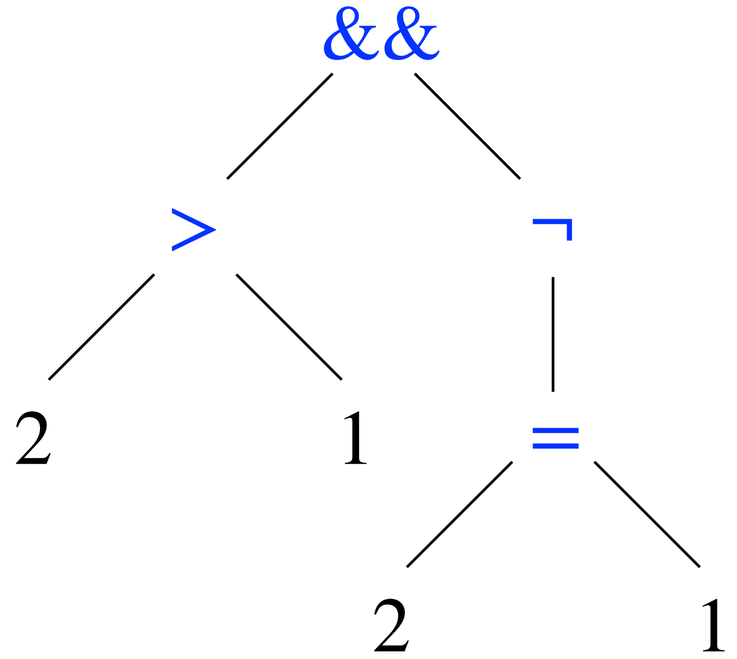
$$3 + (5 - 2) \times 1$$

Boolean Expressions

$$
\begin{aligned}
b := \ & \text{true} \\
& \mid \text{false} \\
& \mid a = a \\
& \mid a \neq a \\
& \mid a \leq a \\
& \mid a > a \\
& \mid \neg b \\
& \mid b \,\&\&\, b
\end{aligned}
$$

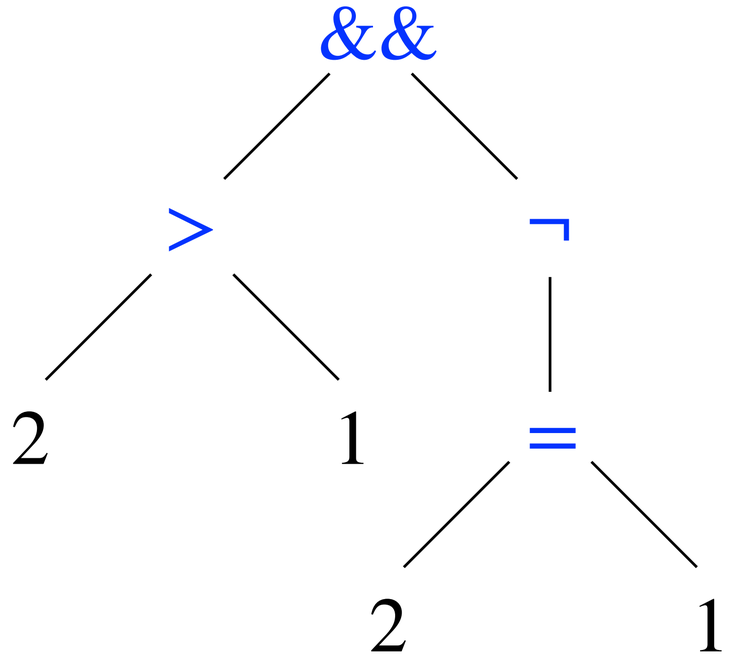
Boolean Expressions

$$
\begin{aligned}
b := \ & \text{true} \\
& \mid \text{false} \\
& \mid a = a \\
& \mid a \neq a \\
& \mid a \leq a \\
& \mid a > a \\
& \mid \neg b \\
& \mid b \&\& b
\end{aligned}
$$



Boolean Expressions

$$
\begin{aligned}
b := \ & \text{true} \\
& \mid \text{false} \\
& \mid a = a \\
& \mid a \neq a \\
& \mid a \leq a \\
& \mid a > a \\
& \mid \neg b \\
& \mid b \&\& b
\end{aligned}
$$



$$2 > 1 \&\& \neg(2 = 1)$$

Evaluating Expressions

Quiz 1

```
Inductive aexp : Type :=
  | ANum (n : nat)
  | APlus (a1 a2 : aexp)
  | AMinus (a1 a2 : aexp)
  | AMult (a1 a2 : aexp).
```

https://etc.ch/FPsM

What does the following expression evaluate to?

```
aeval (APlus (ANum 3) (AMinus (ANum 4) (ANum 1)))
```

1) `true`
2) `false`
3) 0
4) 3
5) 6

```
Fixpoint aeval (a : aexp) : nat :=
  match a with
  | ANum n => n
  | APlus  a1 a2 => (aeval a1) + (aeval a2)
  | AMinus a1 a2 => (aeval a1) - (aeval a2)
  | AMult  a1 a2 => (aeval a1) * (aeval a2)
  end.
```

More Tactics… and Tacticals!

Expression Evaluation as a Relation

$e \Rightarrow n$ "expression e evaluates to number n"

$$\frac{}{n \Rightarrow n}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 + e_2 \Rightarrow n_1 + n_2}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 \times e_2 \Rightarrow n_1 \times n_2}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 - e_2 \Rightarrow n_1 - n_2}$$

Computational vs. Relational Definitions

Expression Evaluation as a Relation

$e \Rightarrow n$ "expression e evaluates to number n"

$$\frac{}{n \Rightarrow n} \qquad \frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 + e_2 \Rightarrow n_1 + n_2}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 \times e_2 \Rightarrow n_1 \times n_2} \qquad \frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 - e_2 \Rightarrow n_1 - n_2}$$

Expression Evaluation as a Relation

$e \Rightarrow n$ "expression e evaluates to number n"

$$\frac{}{n \Rightarrow n}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 + e_2 \Rightarrow n_1 + n_2}$$

makes relation partial

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2 \qquad n_2 > 0 \qquad n_2 \times n_3 = n_1}{e_1 \div e_2 \Rightarrow n_3}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 \times e_2 \Rightarrow n_1 \times n_2}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 - e_2 \Rightarrow n_1 - n_2}$$

Expression Evaluation as a Relation

$e \Rightarrow n$ "expression e evaluates to number n"

$$\frac{}{n \Rightarrow n}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 + e_2 \Rightarrow n_1 + n_2}$$

makes relation partial

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2 \qquad n_2 > 0 \qquad n_2 \times n_3 = n_1}{e_1 \div e_2 \Rightarrow n_3}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 \times e_2 \Rightarrow n_1 \times n_2}$$

$$\frac{e_1 \Rightarrow n_1 \qquad e_2 \Rightarrow n_2}{e_1 - e_2 \Rightarrow n_1 - n_2}$$

$$\frac{}{? \Rightarrow n}$$

makes relation non-deterministic

Expressions with Variables

Arithmetic Expressions with Variables

$$
\begin{aligned}
a &:= n \\
&\quad \mid x \\
&\quad \mid a + a \\
&\quad \mid a - a \\
&\quad \mid a \times a \\
n &\in \mathbb{N} \\
x &\in ?
\end{aligned}
$$

Arithmetic Expressions with Variables

$$
\begin{aligned}
a &:= n \\
&\quad \mid x \\
&\quad \mid a + a \\
&\quad \mid a - a \\
&\quad \mid a \times a \\
n &\in \mathbb{N} \\
x &\in ?
\end{aligned}
$$

$$\texttt{aeval}\ x = ?$$

Arithmetic Expressions with Variables

$$
\begin{aligned}
a &:= n \\
&\quad \mid x \\
&\quad \mid a + a \\
&\quad \mid a - a \\
&\quad \mid a \times a \\
n &\in \mathbb{N} \\
x &\in ?
\end{aligned}
$$

$$\mathit{aeval}\ x = ?$$

st : state

```
Definition state := total_map nat.

Definition total_map (A : Type) := string -> A.
```

Maps

```
Definition total_map (A : Type) := string -> A.
```

Maps

```
Definition total_map (A : Type) := string -> A.
```

```
t_empty: forall {A:Type}, A -> total_map A
```

```
Definition t_empty {A : Type} (v : A) : total_map A :=
  (fun _ => v).
```

The empty map maps all keys to a default value

Maps

```
Definition total_map (A : Type) := string -> A.
```

The empty map maps all keys to a default value

```
t_empty: forall {A:Type}, A -> total_map A
```

```
Definition t_empty {A : Type} (v : A) : total_map A :=
  (fun _ => v).
```

Update returns a map with an added key-value-pair

```
t_update: forall {A:Type}, total_map A -> string -> A -> total_map A
```

```
Definition t_update {A : Type} (m : total_map A)(x : string) (v : A) :=
  fun x' => if String.eqb x x' then v else m x'.
```

Maps

```
Definition total_map (A : Type) := string -> A.
```

The empty map maps all keys to a default value

```
t_empty: forall {A:Type}, A -> total_map A
```

```
Definition t_empty {A : Type} (v : A) : total_map A :=
  (fun _ => v).
```

Update returns a map with an added key-value-pair

```
t_update: forall {A:Type}, total_map A -> string -> A -> total_map A
```

```
Definition t_update {A : Type} (m : total_map A)(x : string) (v : A) :=
  fun x' => if String.eqb x x' then v else m x'.
```

```
Notation "x '!->' v ';' m" := (t_update m x v)

Notation "'_' '!->' v" := (t_empty v)
```

Maps

```
Definition total_map (A : Type) := string -> A.
```

The empty map maps all keys to a default value

```
t_empty: forall {A:Type}, A -> total_map A
```

```
Definition t_empty {A : Type} (v : A) : total_map A :=
  (fun _ => v).
```

Update returns a map with an added key-value-pair

```
t_update: forall {A:Type}, total_map A -> string -> A -> total_map A
```

```
Definition t_update {A : Type} (m : total_map A)(x : string) (v : A) :=
  fun x' => if String.eqb x x' then v else m x'.
```

```
Notation "x '!->' v ';' m" := (t_update m x v)

Notation "'_' '!->' v" := (t_empty v)
```

```
Definition examplemap' :=
  ( "bar" !-> true;
    "foo" !-> true;
    _     !-> false
  ).
```

IMP - simple imperative programs

IMP - Syntax

c := `skip`
| x := a
| c ; c
| `if` b `then` c `else` c `end`
| `while` b `do` c `end`

IMP - Syntax

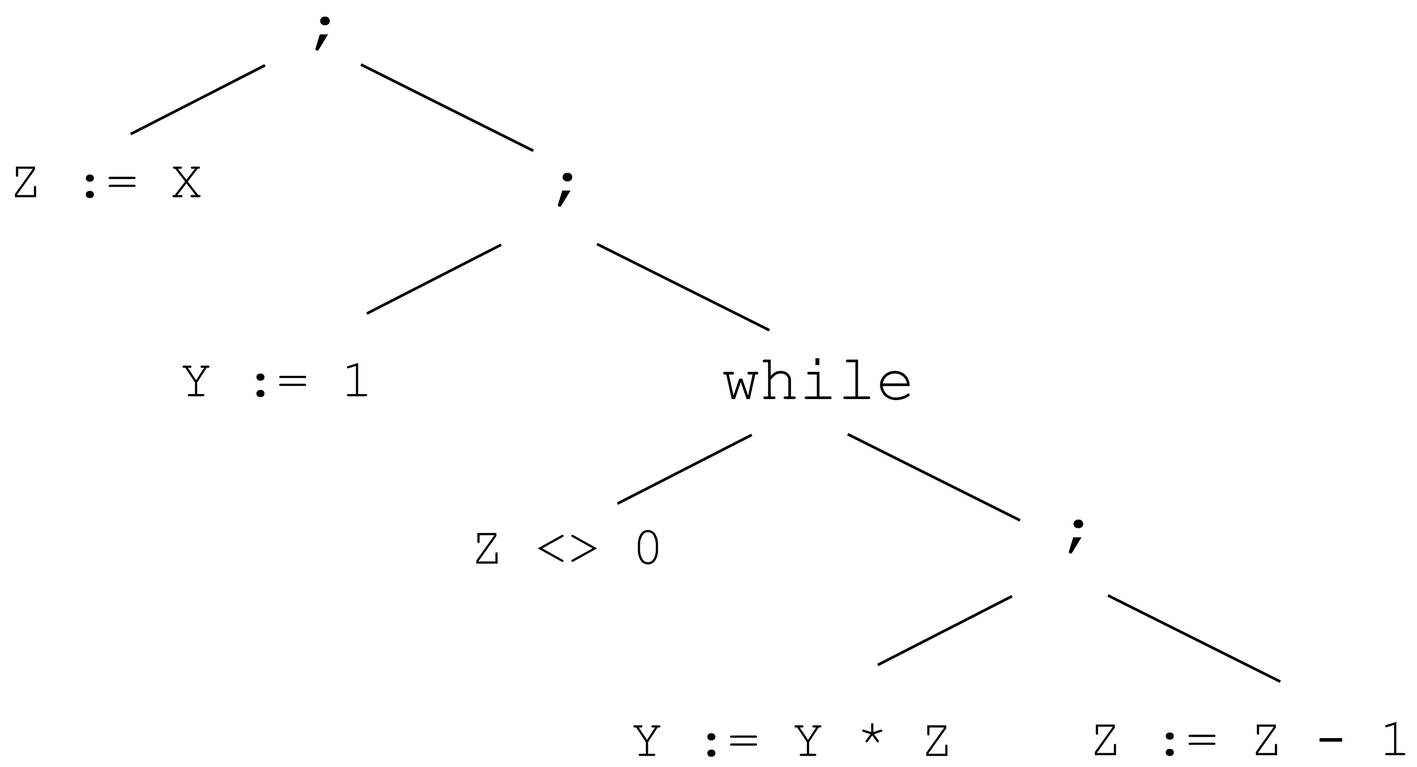
c := `skip`
| x := a
| c ; c
| `if` b `then` c `else` c `end`
| `while` b `do` c `end`

```
Z := X;
Y := 1;
while Z <> 0 do
  Y := Y * Z;
  Z := Z - 1
end
```

IMP - Syntax

c := skip
| $x := a$
| $c ; c$
| if b then c else c end
| while b do c end

```
Z := X;
Y := 1;
while Z <> 0 do
  Y := Y * Z;
  Z := Z - 1
end
```



IMP - Semantics

IMP - Semantics

E_Skip $\dfrac{}{st \xrightarrow{\texttt{skip}} st}$

IMP - Semantics

$$\textbf{E\_Skip}\ \frac{}{st \xrightarrow{\texttt{skip}} st} \qquad \textbf{E\_Asgn}\ \frac{a \Rightarrow_{\texttt{st}} n}{st \xrightarrow{x := a} st[x \mapsto n]}$$

IMP - Semantics

$$\textbf{E\_Skip}\ \frac{}{st \xrightarrow{\texttt{skip}} st} \qquad \textbf{E\_Asgn}\ \frac{a \Rightarrow_{\texttt{st}} n}{st \xrightarrow{x := a} st[x \mapsto n]} \qquad \textbf{E\_Seq}\ \frac{st \xrightarrow{c_1} st' \qquad st' \xrightarrow{c_2} st''}{st \xrightarrow{c_1 ; c_2} st''}$$

IMP - Semantics

E_Skip
$$\frac{}{st \xrightarrow{\texttt{skip}} st}$$

E_Asgn
$$\frac{a \Rightarrow_{\texttt{st}} n}{st \xrightarrow{x := a} st[x \mapsto n]}$$

E_Seq
$$\frac{st \xrightarrow{c_1} st' \qquad st' \xrightarrow{c_2} st''}{st \xrightarrow{c_1 ; c_2} st''}$$

E_IfTrue
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{true} \qquad st \xrightarrow{c_1} st'}{st \xrightarrow{\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2 \texttt{ end}} st'}$$

E_IfFalse
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{false} \qquad st \xrightarrow{c_2} st'}{st \xrightarrow{\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2 \texttt{ end}} st'}$$

IMP - Semantics

E_Skip
$$\frac{}{st \xrightarrow{\texttt{skip}} st}$$

E_Asgn
$$\frac{a \Rightarrow_{\texttt{st}} n}{st \xrightarrow{x := a} st[x \mapsto n]}$$

E_Seq
$$\frac{st \xrightarrow{c_1} st' \qquad st' \xrightarrow{c_2} st''}{st \xrightarrow{c_1 ; c_2} st''}$$

E_IfTrue
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{true} \qquad st \xrightarrow{c_1} st'}{st \xrightarrow{\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2 \texttt{ end}} st'}$$

E_IfFalse
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{false} \qquad st \xrightarrow{c_2} st'}{st \xrightarrow{\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2 \texttt{ end}} st'}$$

E_WhileTrue
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{true} \qquad st \xrightarrow{c} st' \qquad st' \xrightarrow{\texttt{while } b \texttt{ do } c \texttt{ end}} st''}{st \xrightarrow{\texttt{while } b \texttt{ do } c \texttt{ end}} st''}$$

E_WhileFalse
$$\frac{b \Rightarrow_{\texttt{st}} \texttt{false}}{st \xrightarrow{\texttt{while } b \texttt{ do } c \texttt{ end}} st}$$

Quiz 2

$$\textbf{E\_Skip}\ \frac{}{st \xrightarrow{\texttt{skip}} st} \qquad \textbf{E\_Seq}\ \frac{st \xrightarrow{c_1} st' \qquad st' \xrightarrow{c_2} st''}{st \xrightarrow{c_1 ; c_2} st''}$$

https://etc.ch/FPsM

Is the following proposition provable?

```
∀ (c : com) (st st' : state),
    st =[ skip ; c ]=> st' →
    st =[ c ]=> st'
```

Summary

Syntax for expressions & programs

$$a := n \mid a + a \mid a - a \mid a \times a$$

$$n \in \mathbb{N}$$

$$c := \texttt{skip} \mid x := a \mid c ; c \mid \texttt{if } b \texttt{ then } c \texttt{ else } c \texttt{ end} \mid \texttt{while } b \texttt{ do } c \texttt{ end}$$

Computational evaluation

```
Fixpoint aeval (a : aexp) : nat :=
```

Relational Semantics

$$\frac{b \Rightarrow \texttt{false} \qquad st \xrightarrow{c_2} st'}{st \xrightarrow{\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2 \texttt{ end}} st'}$$

Tacticals

;

try

repeat